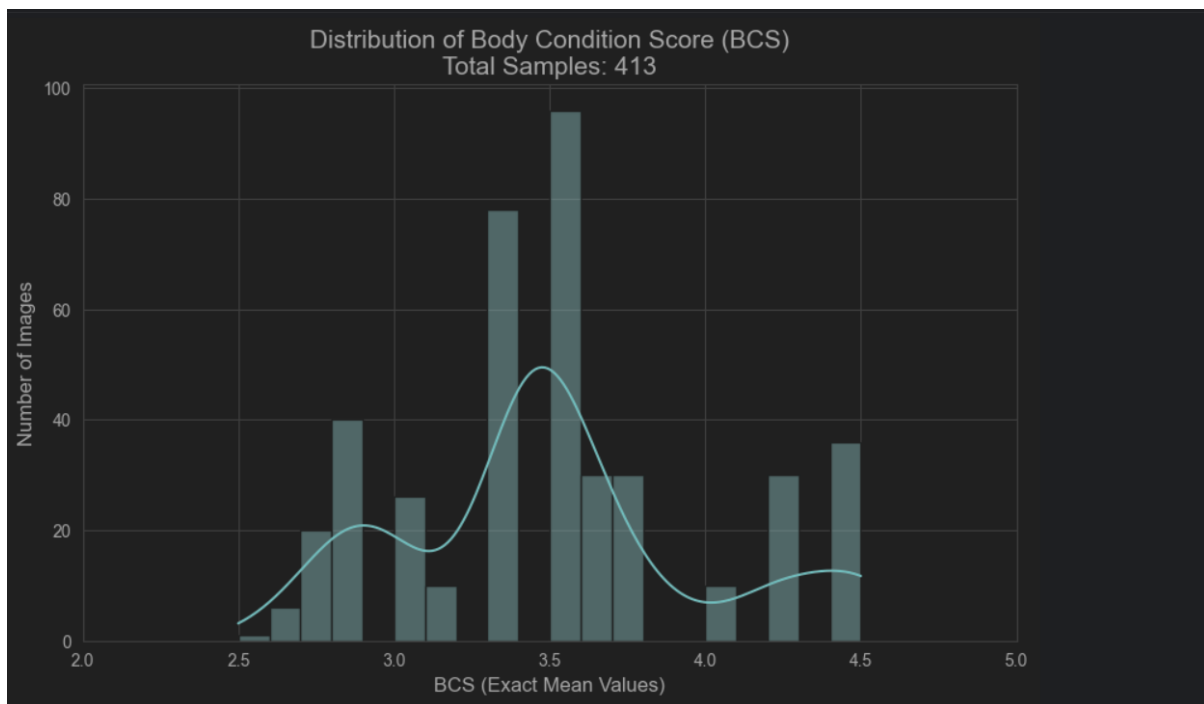


انتخاب دیتاست مناسب	۱
مراحل پیاده سازی	۴
۱. خواندن دیتاست	۴
۲. تقسیم بندی داده ها	۴
۳. transformers	۴
۴. ساخت دیتاست	۵
۵. ساخت دیتالودر	۵
۶. استفاده از مدل RestNet	۵
۷. train کردن مدل	۵
۸. پیدا کردن هایپرپارامتر ها	۶
۹. تست کردن مدل آموزش دیده	۶

انتخاب دیتاست مناسب

این مرحله دیتاست های زیادی بررسی و مشاهده شد. نکته ای که بیشتر دیتاست ها داشتند این بود که به جای اینکه RGB باشند RBGD بودند یعنی چهار کانال داشتند و بعد عمق را هم در نظر می گرفتند، با توجه به اینکه هدف ما ترنسفر لرنینگ با استفاده از معماری ResNet18 بود استفاده از عکس های RBGD مناسب نبود زیرا این معماری انتظار ورودی با ۳ کانال را دارد و در صورت تغییر لایه اول برای دریافت ورودی با ۴ کانال امکان استفاده از وزن های از قبل آموزش دیده از بین می رود. مشکل دیگری که بعضی از دیتاست ها داشتند حجم بالای آنها و عدم دسترسی به سخت افزار قوی در ابتدای شروع پروژه بود. در نهایت دیتاست مورد نظر از سایت <https://iplab.dmi.unict.it/legacy/BCS/> انتخاب شد که حدود ۴۰۰ عکس دارد. مشکلی که این سایت داشت این بود که دیتاست به صورت یک فایل مجزا قابل دانلود نبود و مجبور به خارج کردن عکس ها با کد از این سایت شدم که در فایل `collect_dataset.py` کدش موجود می باشد.

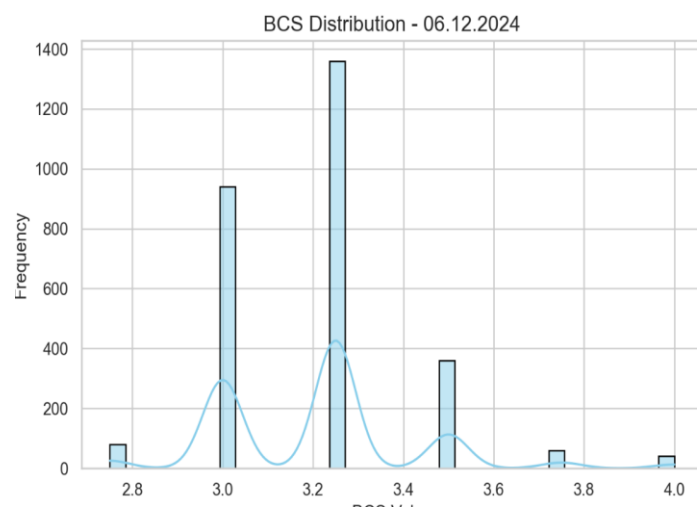
توزیع داده های این دیتاست در عکس زیر قابل مشاهده هست:

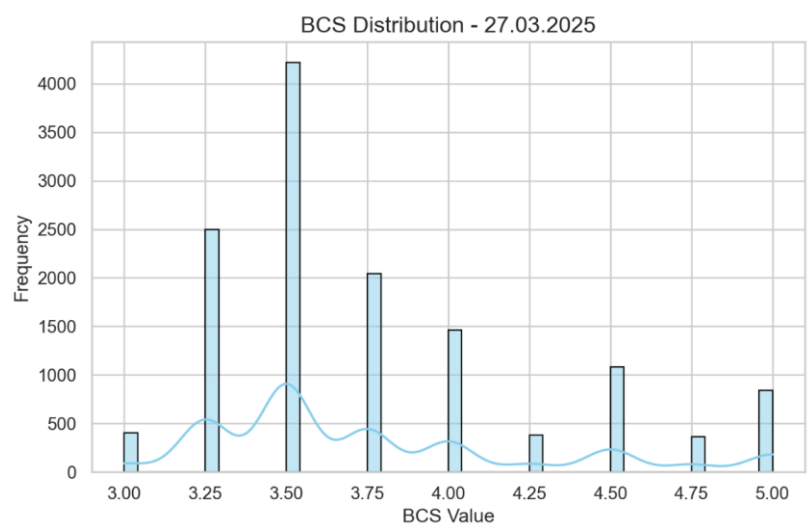
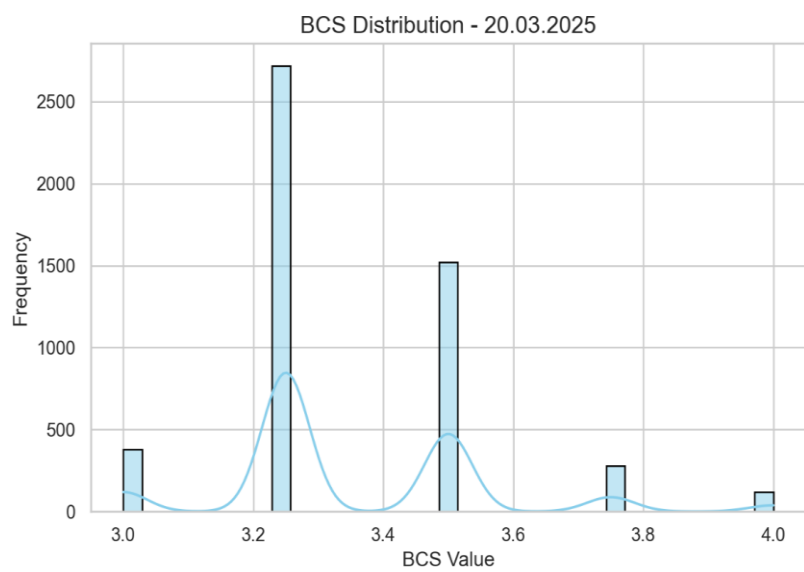
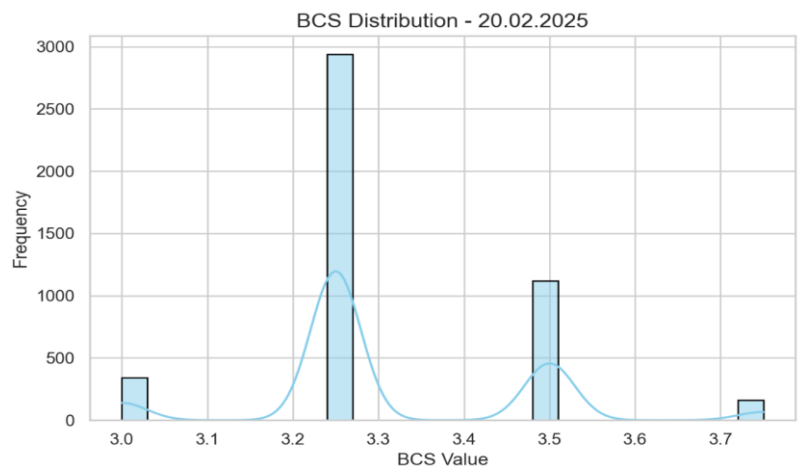


همانطور که مشخص است داده ها به طرز واضحی نامتوازن هستند. در اینجا برای افزایش تنوع نمونه‌های آموزشی و کاهش احتمال overfitting از data augmentation استفاده شد. و البته به دلیل اینکه مقادیر خروجی تقریباً به صورت پیوسته هست بنابراین مسئله به روش رگرسیون حل شد.

یکی دیگر از دیتاست‌هایی که بررسی شد دیتاست سایت <https://zenodo.org/records/20290988> بود این دیتاست هم تصاویر RGB و هم تصاویر RGBD داشت با توجه به شیتی که برای دیتاست‌ها گذاشته بودند توزیع دیتاست‌های مختلف پیدا شد.

در زیر توزیع‌های دیتاست‌ها را می‌گذارم و اسم دیتاست‌ها هم بالای تصاویر قابل مشاهده است:





که البته به دلیل حجم بالا دانلود نشد.

مراحل پیاده سازی

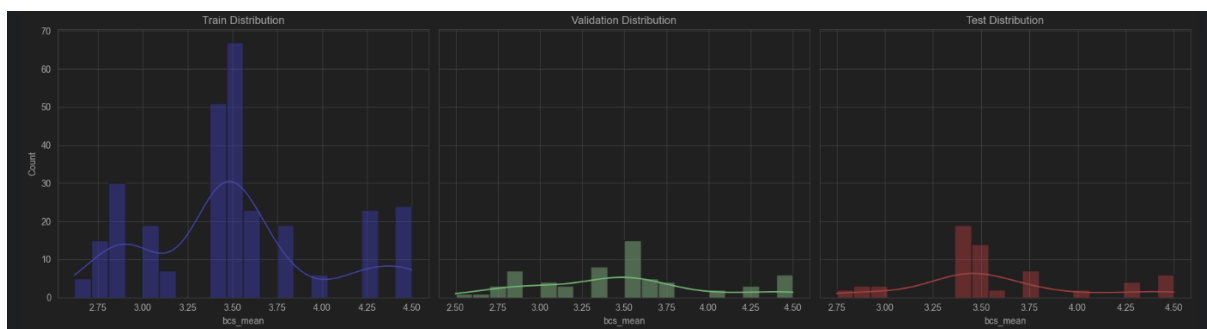
۱. خواندن دیتاست

مرحله اول خواندن دیتاست و بررسی توزیع داده ها بود. در مرحله بعدی فایل ها و عکس های خراب باید حذف میشدند به این صورت که اگر فرمت فایل مورد نظر جزو فرمت های قابل قبول برای عکس نبود یا اینکه عکس کاملاً سیاه یا کاملاً سفید بود (با میانگین گرفتن روی پیکسل ها و مقایسه با **threshold** های مورد نظر برای رنگ های سیاه و سفید مشخص شد). حذف میشدند.

۲. تقسیم بندی داده ها

مرحله دوم تقسیم بندی داده ها به سه دسته **train, test, validation** بود که به ۷۰ درصد داده ترین و ۱۵ درصد داده تست و ولیدیشن تقسیم بندی شد. این تقسیم بندی لازم بود که حتماً به صورت **stratify** انجام بشود که توزیع داده ها در هر سه دسته ثابت باقی بماند و با توجه به ماهیت پیوسته داده ها ۴ دسته، داده ها در ۴ دسته مختلف گروه بندی شدند و بعد تقسیم بندی به صورت **stratify** روی این دسته ها انجام شد.

توزیع نهایی به صورت زیر است:



۳. transformers

مرحله سوم بخش **transformer** ها بود زیرا برای کار کردن با شبکه های عصبی لازم هست که عکس ها تبدیل به **tensor** بشوند. در اینجا دو نوع ترنسفورمر داریم یکی برای داده های آموزش که باید **data augmentation** هم روی آنها اعمال شود و یکی برای داده های تست و ولیدیشن که صرفاً باید ریسایز، تبدیل به تانسور و در نهایت با توجه به میانگین و انحراف معیار مدل **ImageNet** نرمال شوند.

Data augmentation که من اعمال کردم به صورت است که با احتمال ۵۰ درصد عکس را ۱۸۰ درجه به صورت افقی قرینه میکند. و تصویر را در مرکز خودش تا ۱۰ درجه به چپ یا تا ۱۰ درجه به راست میچرخاند و در نهایت ۱۰ درصد تغییر در نور و کنتراست ایجاد میکند.

۴. ساخت دیتاست

مرحله چهارم پیاده سازی دیتاست بود که برای استفاده در ساخت دیتالودر ها ضروری بود. در اینجا در متد `getitem` عکس و لیبل جدا میشد و روی عکس ترنسفورمر مد نظر اعمال میشد و لیبل هم تبدیل به تنسور میشد.

۵. ساخت دیتالودر

مرحله پنجم ساخت دیتالودر بود که دیتاست های مدنظر (ترین و تست و ولیدیشن) را به بچ های ۳۲ تایی تبدیل میکرد که برای ترین کردن مدل ضروری بود زیرا حافظه اجازه لود کردن کل دیتاست در رم را به ما نمیدهد.

۶. استفاده از مدل RestNet

مرحله ششم لود کردن مدل `RestNet` بود در کلاس `BCSResNet18` از کلاس `Module` ارث بری میشود. در این کلاس وزن های مدل لود شد و مدل ساخته شد و سپس لایه آخر یعنی `fc` به `Linear` تغییر پیدا کرد زیرا خود این مدل در لایه آخر به ۱۰۰۰ کلاس تبدیل میشود اما خروجی مورد نظر ما رگرسیون هست پس باید یک خروجی عددی به ما بدهد. در بخش بعدی تمام لایه های مدل فریز شد به این معنا که در زمان ترین مقدارشان تغییر نکنند زیرا مدل از قبل آموزش دیده است و صرفا لایه های آخر باید آموزش ببینند در اینجا مم سه حالت برای آموزش قرار دادم: آموزش لایه `fc`، آموزش لایه ۴ و لایه `fc` و در نهایت آموزش لایه ۳ و لایه ۴ و لایه `fc` که در ورودی این کلاس باید مشخص کنیم که کدام لایه ها `trainable` باشند.

۷. train کردن مدل

در این مرحله کلاس `Trainer` پیاده سازی شد. این کلاس اسم های `optimizer` و `criterion` که همان تابع هزینه هست را میگرد پس باید تبدیل به مقادیر واقعی کلاس `nn` شود پس متد های `build_optimizer` و `build_criterion` پیاده سازی شد.

در متد `train_one_eoch` قرار است که مراحل گفته شده انجام شود: اولین مرحله فوروارد است که در واقع پیش بینی مدل فعلی برای ورودی هاست. دومین مرحله محاسبه مقدار لاس بر اساس معیار داده شده در ورودی کلاس است. در مرحله `backward`، با استفاده از زنجیره مشتق گیری، گرادیان تابع `loss` نسبت به پارامترهای `trainable` مدل محاسبه می شود. سپس `optimizer` با استفاده از این گرادیان ها پارامترهای قابل آموزش را به روزرسانی می کند. در نهایت مقادیر میانگین `loss, absolute error` برای آن بچ محاسبه میشوند. و در آخر میانگین کل برای همه نمونه ها محاسبه میشود و آموزش یک اپوک به انتها میرسد.

در متد `validate` قرار است که مقدار لاس داده های ولیدیشن برای اپوک که در مرحله قبل کمی آموزش دیده محاسبه شود اینکار کمک میکند در آینده با استفاده از `early stopping` از `overfitting` جلوگیری شود. در اینجا مدل را به حالت `eval` میبریم که باعث میشود آپشن هایی که برای آموزش داریم مانند `dropout` و ... اعمال نشوند. سپس مقدار لاس برای یک اپوک مانند بخش قبلی محاسبه میشود.

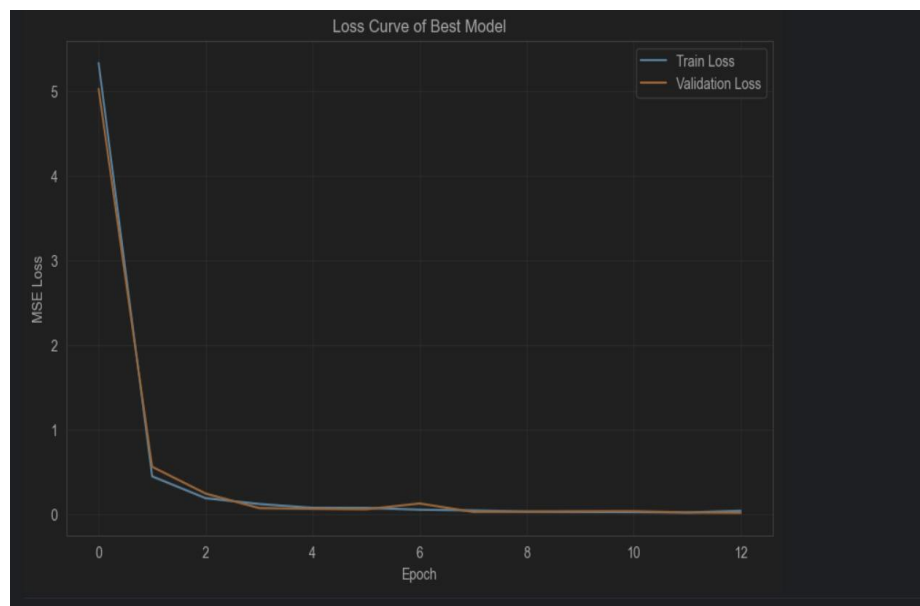
و در نهایت متد فیت که به تعداد اپوک های خواسته شده متد `train_one_epoch` را صدا میزند و سپس متد `validate` را صدا میزند و اطلاعات هر اپوک را ذخیره کرده و در نهایت `early stopping` را اعمال میکند.

۸. پیدا کردن هایپرپارامترها

در اینجا هایپر پارامترها شامل لایه های `trainable`، `optimizer` و `learning rates` است در حالت های مختلف تست میشوند و بر اساس `val_loss` بهترین مدل انتخاب میشود. و در نهایت نمودار لاس برای بهترین مدل رسم شد. در نهایت بهترین مدل به صورت زیر درآمد:

```
trainable_layers': 'layer3_layer4_fc', 'optimizer': 'adam', 'learning_rate': 0.0003'
```

Val loss: 0.028262129233729456



۹. تست کردن مدل آموزش دیده

در این بخش هم دوباره مدل به حالت `eval` برده شد و مقدار `prediction` مدل برای داده های تست محاسبه شد و با مقدار واقعی مقایسه شد. معیار های مقایسه به صورت زیر شدند:

MSE: 0.0084915179759264

RMSE: 0.09214943285732365

MAE: 0.07298927754163742

و نمودار actual , predicted به صورت زیر در آمد:

